

C++で形式的証明

- C++で形式的証明
 - 1. 概要
 - 2. 命題論理の公理・推論規則
 - 3. 命題論理の実装
 - 3.1. 基本方針
 - 3.2. 基底クラス
 - 3.3. `False`
 - 3.4. `Not`
 - 3.5. `And`
 - 3.6. `Or`
 - 3.7. `Impl`
 - 3.8. `Equiv`
 - 3.9. `Prop`
 - 3.10. 補足
 - 3.10.1. コピーコンストラクタについて
 - 3.10.2. ラムダ式の副作用について
 - 4. 命題論理の証明例
 - 4.1. 対偶律
 - 4.2. ド・モルガンの法則
 - 4.3. 二重否定の導入・除去
 - 4.4. 同一律
 - 5. 述語論理の公理・推論規則
 - 6. 述語論理の実装
 - 6.1. 基本方針
 - 6.2. メタ関数
 - 6.3. 基底クラス
 - 6.4. `All`
 - 6.5. `Exist`
 - 6.6. `Pred`
 - 6.7. `Bound`
 - 6.8. 補足
 - 6.8.1. 命題としての区別について
 - 6.8.2. 束縛変数に重複がある命題について
 - 7. 述語論理の証明例
 - 7.1. ド・モルガンの法則 (述語論理版)
 - 7.2. `Exist` と `All` の順序交換
 - 7.3. `Exist` の `And` に対する分配
 - 7.4. `All` の `Or` に対する分配
 - 8. 実装の全体像
 - 9. アピールポイント

1. 概要

本レポートでは述語論理の形式的証明ツールをC++で実装します。形式的証明のコンパイラをC++で書くという意味ではなく、C++の言語機能をそのまま形式的証明の構文に利用し、C++のコードとして形式的証明をコンパイルすることを目指します。正しい証明に対してコンパイルが通るようにすることはもちろん、常識的な範囲の間違いを含む証明に対してコンパイルエラーを出すことも目標にします。

述語論理自体は講義で直接扱われてはいませんが、講義に登場した集合、写像、関係、濃度といった概念は、すべて述語論理の厳密な定式化の下で成り立っています。また、「任意の・・・」「・・・が存在する」という言い回しなど、述語論理の概念が暗黙的に使われている場面も多数ありました。したがって、述語論理も講義で扱われた概念の一つとして認めてよいと考えています。

本レポートで提示するコードの動作確認はすべてg++で行っています。他の環境で動くかは確かめていません。

実際にコードを動かすときは次のコンパイルオプションを付けてください。

```
-fsyntax-only
-std=c++23
```

- -fsyntax-only : 構文チェックのみを行うオプションです。実行ファイルが出力されなくなります。
- -std=c++23 : バージョンとしてC++23を指定します。

2. 命題論理の公理・推論規則

まずは述語論理の土台である命題論理を構築します。今回は記号として $\perp, \neg, \wedge, \vee, \rightarrow, \leftrightarrow$ を使うこととし、公理および推論規則として以下を採用します。

- 公理
 - 排中律

$$P \vee \neg P$$

- 推論規則
 - 爆発律

$$\frac{\perp}{P}$$

- $\neg$ 導入則

$$\frac{\begin{array}{c} [P] \\ \vdots \\ \perp \end{array}}{\neg P}$$

- $\neg$ 除去則

$$\frac{P \quad \neg P}{\perp}$$

- $\wedge$ 導入則

$$\frac{P \quad Q}{P \wedge Q}$$

- $\wedge$ 除去則

$$\frac{P \wedge Q}{P}$$

$$\frac{P \wedge Q}{Q}$$

- $\vee$ 導入則

$$\frac{P}{P \vee Q}$$

$$\frac{Q}{P \vee Q}$$

- $\vee$ 除去則

$$\frac{\begin{array}{ccc} [P] & [Q] \\ \vdots & \vdots \\ P \vee Q & R & R \end{array}}{R}$$

- $\rightarrow$ 導入則

$$\frac{\begin{array}{c} [P] \\ \vdots \\ Q \end{array}}{P \rightarrow Q}$$

- $\rightarrow$ 除去則

$$\frac{P \quad P \rightarrow Q}{Q}$$

- $\leftrightarrow$ 導入則

$$\frac{\begin{array}{cc} [P] & [Q] \\ \vdots & \vdots \\ Q & P \end{array}}{P \leftrightarrow Q}$$

- $\leftrightarrow$ 除去則

$$\frac{P \quad P \leftrightarrow Q}{Q}$$

$$\frac{Q \quad P \leftrightarrow Q}{P}$$

3. 命題論理の実装

3.1. 基本方針

プログラミング上で数学的な証明を取り扱うために、次のような対応関係を考えます。

数学	プログラミング (C++)
命題	クラス
証明	オブジェクト
証明する	クラスのコンストラクタを呼ぶ
証明した命題を使う	オブジェクトからメンバ関数を呼ぶ
仮定する	仮定を引数として受け取るラムダ式を定義する
仮定付きで証明する	仮定を引数として受け取るラムダ式から目的の命題を返す

この対応関係の下で、論理記号 $\perp, \neg, \wedge, \vee, \rightarrow, \leftrightarrow$ を以下のようなクラスとして実装することにします。

論理記号	コンストラクタ	メンバ
$\perp$	なし	任意の命題を返す (爆発律)
$\neg P$	P を受け取って $\perp$ を返すラムダ式を要求 ($\neg$ 導入則)	P を受け取って $\perp$ を返す ($\neg$ 除去則)
$P \wedge Q$	P, Q を両方とも要求 ($\wedge$ 導入則)	P, Q をそれぞれ返す ($\wedge$ 除去則)
$P \vee Q$	P または Q の一方を要求 ($\vee$ 導入則)	「 P を受け取って R を返すラムダ式」「 Q を受け取って R を返すラムダ式」の2つを受け取って R を返す ($\vee$ 除去則)
$P \rightarrow Q$	P を受け取って Q を返すラムダ式を要求 ($\rightarrow$ 導入則)	P を受け取って Q を返す ($\rightarrow$ 除去則)
$P \leftrightarrow Q$	「 P を受け取って Q を返すラムダ式」「 Q を受け取って P を返すラムダ式」の2つを要求 ($\leftrightarrow$ 導入則)	P, Q の一方を受け取って他方を返す ($\leftrightarrow$ 除去則)

$P \vee \neg P$ および $\neg P \vee P$ はデフォルトコンストラクタを持つものとします。 (排中律)
命題変数 $P, Q, R, \dots$ もクラスとして実装します。したがって $\perp$ 以外の論理記号はクラステンプレートとなります。

3.2. 基底クラス

除去則を実装するためには、命題クラスどうしてオブジェクトをやりとりさせる必要があります。そこで、すべての命題クラスの基底クラス `PropBase` を定義し、その中で命題クラスのオブジェクトを一括管理する

ことにします。

```
class PropBase;

template <class P>
concept PropType = std::is_base_of_v<PropBase, P>;

class PropBase {
protected:
    consteval PropBase() {}

    template <PropType P>
    static constexpr P object = P();
};
```

`PropBase` を継承した命題クラス `P` は、`PropBase` に対してデフォルトコンストラクタ `P()` へのアクセスを許可しつつ、`PropBase::object` を介して必要なオブジェクトを入手するものとします。

`PropType` は命題クラスを表すコンセプトです。`std::is_base_of_v` を用いて `PropBase` の派生クラスであるか判定しています。

`PropBase()` に付いている `consteval` はコンパイル時評価を強制するための修飾子です。`constexpr` と異なり、コンパイル時評価できなかった場合にはエラーが出ます。本レポートの形式的証明ツールはコンパイルのみで完結するので、すべての関数に例外なく `consteval` を付けています。

3.3. False

```
class False final : PropBase {
public:
    template <PropType P>
    consteval P explode() const {
        return PropBase::object<P>;
    }

private:
    friend class PropBase;
    consteval False() {}
};
```

`False` クラスは論理記号 $\perp$ に対応します。

メンバ関数 `explode` は爆発律を実装したものです。一度 `False` のオブジェクトを作ることができれば、`explode` を使ってあらゆる命題を証明できます。

`private:` セクション内に書かれている `friend` 宣言とデフォルトコンストラクタは、前節で説明した通り `PropBase` 内で `False` のオブジェクトを作れるようにしたものです。この部分の実装はすべての命題クラスで共通なので、以降説明を省略します。

3.4. Not

```
template <PropType P>
class Not final : PropBase {
public:
    consteval Not(auto f) {
        False q = f(PropBase::object<P>);
    }

    consteval False operator()(P) const {
        return PropBase::object<False>;
    }

private:
    friend class PropBase;
    consteval Not() {}
};
```

Not クラステンプレートは論理記号 $\neg$ に対応します。

コンストラクタ `Not(auto f)` は $\neg$ 導入則を表しています。ラムダ式 `f` に `P` クラスのオブジェクトを渡して `False` クラスのオブジェクトが返ってくるか検証します。

`operator()` は $\neg$ 除去則を表しています。 `P` クラスのオブジェクトを受け取って `False` クラスのオブジェクトを返します。

※ `Not<P>` の機能は `Impl<P, False>` と本質的に同じなので、エイリアステンプレートを用いて次のように書いても `Not` の実装として十分です。(`Impl` の実装については3.7節を参照してください)

```
template <class P>
using Not = Impl<P, False>
```

しかしながら、間違った証明を書いたとき、エラーメッセージではエイリアスが元の名前に展開されるので、ただでさえ読みにくいエラーメッセージがさらに読みにくくなってしまいます。本レポートではこれを避けるためエイリアスを積極的に使っていません。

3.5. And

```
template <PropType P, PropType Q>
class And final : PropBase {
public:
    consteval And(P, Q) {}

    const P left = PropBase::object<P>;
    const Q right = PropBase::object<Q>;

private:
```

```
friend class PropBase;
constexpr And() {}
};
```

And クラステンプレートは論理記号 $\wedge$ に対応します。

コンストラクタ `And(P, Q)` は $\wedge$ 導入則を表しています。 `P`, `Q` のオブジェクトから `And<P, Q>` のオブジェクトを構築できます。

メンバ変数 `left`, `right` は $\wedge$ 除去則を表しています。

3.6. Or

```
template <PropType P, PropType Q>
class Or final : PropBase {
public:
    constexpr Or() requires (std::same_as<Q, Not<P>> || std::same_as<P, Not<Q>>)
    {}
    constexpr Or(P) {}
    constexpr Or(Q) requires (!std::same_as<P, Q>) {}

    constexpr auto elim(auto f, auto g) const {
        auto rf = f(PropBase::object<P>());
        auto rg = g(PropBase::object<Q>());
        static_assert(std::same_as<decltype(rf), decltype(rg)>);
        return rf;
    }

private:
    friend class PropBase;
    constexpr Or() {}
};
```

Or クラステンプレートは論理記号 $\vee$ に対応します。

デフォルトコンストラクタは排中律を表しています。 `Q` と `Not<P>` がクラスとして等しいとき、または `P` と `Not<Q>` がクラスとして等しいとき、デフォルトコンストラクタを呼び出すことができます。

2, 3番目のコンストラクタは $\vee$ 導入則を表しています。 `P` または `Q` のオブジェクトから `Or<P, Q>` のオブジェクトを構築できます。

メンバ関数 `elim` は $\vee$ 除去則を表しています。2つのラムダ式 `f`, `g` を引数として受け取り、それぞれ `P`, `Q` のオブジェクトを渡して同じクラスのオブジェクトが返ってくるか検証します。戻り値としてそのオブジェクトを返します。

3.7. Impl

```
template <PropType P, PropType Q>
class Impl final : PropBase {
```

```

public:
    consteval Impl(auto f) {
        Q q = f(PropBase::object<P>);
    }

    consteval Q operator()(P) const {
        return PropBase::object<Q>;
    }

private:
    friend class PropBase;
    consteval Impl() {}
};

```

Impl クラステンプレートは論理記号 $\rightarrow$ に対応します。

コンストラクタ **Impl(auto f)** は $\rightarrow$ 導入則を表しています。ラムダ式 **f** に **P** のオブジェクトを渡して **Q** のオブジェクトが返ってくるか検証します。

operator() は $\rightarrow$ 導入則を表しています。**P** のオブジェクトを受け取って **Q** のオブジェクトを返します。

3.8. **Equiv**

```

template <PropType P, PropType Q>
class Equiv final : PropBase {
public:
    consteval Equiv(auto f, auto g) {
        Q q = f(PropBase::object<P>);
        P p = g(PropBase::object<Q>);
    }

    consteval Q operator()(P) const {
        return PropBase::object<Q>;
    }

    consteval P operator()(Q) const requires (!std::same_as<P, Q>) {
        return PropBase::object<P>;
    }

private:
    friend class PropBase;
    consteval Equiv() {}
};

```

Impl クラステンプレートは論理記号 $\leftrightarrow$ に対応します。

コンストラクタ **Equiv(auto f, auto g)** は $\leftrightarrow$ 導入則を表しています。2つのラムダ式 **f**, **g** を受け取り、ラムダ式 **f** に **P** のオブジェクトを渡して **Q** のオブジェクトが返ってくるか、ラムダ式 **g** に **Q** のオブジェクトを渡して **P** のオブジェクトが返ってくるか検証します。

`operator()` は $\leftrightarrow$ 導入則を表しています。 `P`, `Q` のオブジェクトのうち一方を受け取って他方を返します。

3.9. Prop

```
template <size_t id>
class Prop final : PropBase {
private:
    friend class PropBase;
    consteval Prop() {}
};
```

`Prop` クラステンプレートは命題変数 $P, Q, R, \dots$ に対応します。 `id` に異なる非負整数を入れることで命題変数として区別します。 `using` ディレクティブを用いて `using P = Prop<0>;` などのエイリアスを定義することを想定しています。

3.10. 補足

3.10.1. コピーコンストラクタについて

上で示した実装ではコピーコンストラクタをデフォルト定義していましたが、実はこのままだと問題が生じます。例えば次のような初期化がコンパイルを通過します。

```
False fal(fal);
```

このような自己初期化はC++では定義された動作であり、初期化されていない `fal` を `fal` にコピーする挙動になります。これを防ぐには、コピーコンストラクタ内に `fal` のメンバ変数を読み取る処理を書けばよいです。

```
class False final : PropBase {
public:
    consteval False(const False& other) {
        initialized = other.initialized; // 追加
    }

    template <PropType P>
    consteval P explode() const {
        return PropBase::object<P>;
    }

private:
    friend class PropBase;
    consteval False() : initialized(true) {}

    bool initialized;
};
```

`other` が正しく初期化されていない場合、コンパイラが未初期化の `other.initialized` を読み取ったことを検知してエラーを出してくれます。

3.10.2. ラムダ式の副作用について

もう一つ問題があります。`Or` の `elim` メンバ関数および `Impl`, `Equiv` のコンストラクタでは、仮定となる命題を直接ラムダ式に渡しています。

```
consteval auto elim(auto f, auto g) const {
    auto rf = f(PropBase::object<P>);
    auto rg = g(PropBase::object<Q>);
    static_assert(std::same_as<decltype(rf), decltype(rg)>);
    return PropBase::object<decltype(rf)>;
}
```

```
consteval Impl(auto f) {
    Q q = f(PropBase::object<P>);
}
```

```
consteval Equiv(auto f, auto g) {
    Q q = f(PropBase::object<P>);
    P p = g(PropBase::object<Q>);
}
```

この仮定はラムダ式の内部のみで使われることが前提ですが、多少工夫すれば仮定を外部に持ち出すコードも書いてしまいます。例えば次のコードがコンパイルを通過します。

```
False* fal_ptr = nullptr;
Or<False, Not<False>>().elim(
    [&](False fal) -> Or<False, Not<False>> {
        fal_ptr = new False(fal);
        return fal;
    },
    [&](Not<False> not_fal) -> Or<False, Not<False>> {
        return not_fal;
    }
);
False fal = *fal_ptr;
delete fal_ptr;
```

`new` 演算子を `delete` 定義することで上のコードはひとまずコンパイルエラーになります。

```

class False final : PropBase {
public:
    consteval False(const False& other) {
        initialized = other.initialized;
    }

    template <PropType P>
    consteval P explode() const {
        return PropBase::object<P>;
    }

    // 追加
    static void* operator new(size_t) = delete;
    static void* operator new[](size_t) = delete;

private:
    friend class PropBase;
    consteval False() : initialized(true) {}

    bool initialized;
};

```

ところが、ポインタの代わりに共用体を使って初期化を遅延し、`std::construct_at` を使ってコピー構築するとコンパイルエラーを回避できてしまいます。

```

#include <memory>

union Fake {
    char dummy;
    False fal;
    constexpr Fake() : dummy(0) {}
};

consteval False solve() {
    Fake fake;
    Or<False, Not<False>>().elim(
        [&](False fal) -> Or<False, Not<False>> {
            std::construct_at(std::addressof(fake.fal), fal);
            return fal;
        },
        [&](Not<False> not_fal) -> Or<False, Not<False>> {
            return not_fal;
        }
    );
    return fake.fal;
}

int main() {

```

```

    solve();
}

```

定数式の文脈では、あらかじめ確保しておいたメモリ上にオブジェクトを構築すること (配置new) は通常できないのですが、`std::construct_at` はそれを可能にするための特別な組み込み関数が用意されているらしいです。

このような手法をコンパイルエラーにする方法は残念ながら思い付きませんでした。とはいえ、ユーザー側が不注意で `std::construct_at` を使って初期化を試みる状況はまず考えられないので、この問題には目を瞑ることにします。

4. 命題論理の証明例

4.1. 対偶律

$$(P \rightarrow Q) \leftrightarrow (\neg Q \rightarrow \neg P)$$

```

using P = Prop<0>;
using Q = Prop<1>;

consteval Equiv<Impl<P, Q>, Impl<Not<Q>, Not<P>>> solve() {
    return {
        [&](Impl<P, Q> impl_p_q) -> Impl<Not<Q>, Not<P>> {
            return [&](Not<Q> not_q) -> Not<P> {
                return [&](P p) -> False {
                    return not_q(impl_p_q(p));
                };
            };
        },
        [&](Impl<Not<Q>, Not<P>> impl_not_q_not_p) -> Impl<P, Q> {
            return [&](P p) -> Q {
                return Or<Q, Not<Q>>().elim(
                    [&](Q q) -> Q {
                        return q;
                    },
                    [&](Not<Q> not_q) -> Q {
                        return impl_not_q_not_p(not_q)(p).explode<Q>();
                    }
                );
            };
        }
    };
}

int main() {
    solve();
}

```

4.2. ド・モルガンの法則

$$\neg(P \wedge Q) \leftrightarrow \neg P \vee \neg Q$$

```
using P = Prop<0>;
using Q = Prop<1>;

consteval Equiv<Not<And<P, Q>>, Or<Not<P>, Not<Q>>> solve() {
    return {
        [&](Not<And<P, Q>> not_and_p_q) -> Or<Not<P>, Not<Q>> {
            return Or<P, Not<P>>().elim(
                [&](P p) -> Or<Not<P>, Not<Q>> {
                    return Not<Q>([&](Q q) -> False {
                        return not_and_p_q({p, q});
                    });
                },
                [&](Not<P> not_p) -> Or<Not<P>, Not<Q>> {
                    return not_p;
                }
            );
        },
        [&](Or<Not<P>, Not<Q>> or_not_p_not_q) -> Not<And<P, Q>> {
            return [&](And<P, Q> and_p_q) -> False {
                return or_not_p_not_q.elim(
                    [&](Not<P> not_p) -> False {
                        return not_p(and_p_q.left);
                    },
                    [&](Not<Q> not_q) -> False {
                        return not_q(and_p_q.right);
                    }
                );
            };
        }
    };
}

int main() {
    solve();
}
```

$$\neg(P \vee Q) \leftrightarrow \neg P \wedge \neg Q$$

```
using P = Prop<0>;
using Q = Prop<1>;

consteval Equiv<Not<Or<P, Q>>, And<Not<P>, Not<Q>>> solve() {
    return {
        [&](Not<Or<P, Q>> not_or_p_q) -> And<Not<P>, Not<Q>> {
            return {
                [&](P p) -> False {
                    return not_or_p_q(p);
                },
            };
        },
    };
}
```

```

        [&](Q q) -> False {
            return not_or_p_q(q);
        }
    };
},
[&](And<Not<P>, Not<Q>> and_not_p_not_q) -> Not<Or<P, Q>> {
    return [&](Or<P, Q> or_p_q) -> False {
        return or_p_q.elim(
            [&](P p) -> False {
                return and_not_p_not_q.left(p);
            },
            [&](Q q) -> False {
                return and_not_p_not_q.right(q);
            }
        );
    };
}
};
}

int main() {
    solve();
}

```

4.3. 二重否定の導入・除去

$$P \leftrightarrow \neg\neg P$$

```

using P = Prop<0>;

consteval Equiv<P, Not<Not<P>>> solve() {
    return {
        [&](P p) -> Not<Not<P>> {
            return [&](Not<P> not_p) -> False {
                return not_p(p);
            };
        },
        [&](Not<Not<P>> not_not_p) -> P {
            return Or<P, Not<P>>().elim(
                [&](P p) -> P {
                    return p;
                },
                [&](Not<P> not_p) -> P {
                    return not_not_p(not_p).explode<P>();
                }
            );
        }
    };
}

int main() {

```

```

    solve();
}

```

4.4. 同一律

$P \leftrightarrow P$

```

using P = Prop<0>;

consteval Equiv<P, P> solve() {
    return {
        [&](P p) -> P {
            return p;
        },
        [&](P p) -> P {
            return p;
        }
    };
}

int main() {
    solve();
}

```

5. 述語論理の公理・推論規則

述語論理では、命題論理の記号に加えて新たに量化記号 $\forall, \exists$ を導入し、次の推論規則を追加します。

- $\forall$ 導入則

$$\frac{P(a/x)}{\forall x P}$$

- $\forall$ 除去則

$$\frac{\forall x P}{P(t/x)}$$

- $\exists$ 導入則

$$\frac{P(t/x)}{\exists x P}$$

- $\exists$ 除去則

$$\frac{\begin{array}{c} P(a/x) \\ \vdots \\ \exists x P \quad Q \end{array}}{Q}$$

$\forall$ 導入則および $\exists$ 除去則中の a は固有変数条件を満たすものとします。すなわち、次の条件が成り立つものとします。

- $\forall$ 導入の場合： a は $P(a/x)$ が依存する前提に自由変数として現れない。
- $\exists$ 除去の場合： a は Q が依存する $P(a/x)$ 以外の前提に自由変数として現れない。

6. 述語論理の実装

6.1. 基本方針

論理記号 $\forall, \exists$ に対応する命題クラステンプレートの実装方針を以下に示します。

論理記号	コンストラクタ	メンバ
$\forall x P$	テンプレート引数に自由変数 a が渡されたとき $P(a/x)$ を返すラムダ式テンプレートを要求 ($\forall$ 導入則)	自由変数 t をテンプレート引数として受け取って $P(a/x)$ を返す ($\forall$ 除去則)
$\exists x P$	$P(t/x)$ を満たす自由変数 t を要求 ($\exists$ 導入則)	「テンプレート引数に自由変数 a が渡されたとき、 $P(a/x)$ を受け取って Q を返すラムダ式テンプレート」を受け取って Q を返す ($\exists$ 除去則)

6.2. メタ関数

論理記号の実装に入る前に、変数への代入を行うためのメタ関数を定義します。数学では単に $P(a/x)$ と書けばよいですが、プログラミングでは命題 P の述語としての構造に現れるすべての x を洗い出して a に置き換えなければなりません。以下では、`ReplaceType` というメタ関数を定義し、`ReplaceType<T, U, V>` でクラス T の構造に現れる U を V に置き換えたクラスを得ることを目指します。例えば、 T が

```
Hoge<U, Fuga<V, U>, W>
```

であれば `ReplaceType<T, U, V>` は

```
Hoge<V, Fuga<V, V>, W>
```

となります。

大前提として、クラス T には元々のテンプレートがどういう構造だったかという情報が全く含まれていないので、まずは T 自体にそれらの情報を埋め込む必要があります。具体的には、すべての命題クラスおよび変数クラスに対して元々のテンプレートをメンバエイリアステンプレート `Template` に格納し、テンプレート引数の一覧を `std::tuple` の形でメンバエイリアス `TemplateArgs` に格納にします。

And クラステンプレートを例にとると次のようになります。

```
template <PropType P, PropType Q>
class And final : PropBase {
```



```
public:
    template <PropType T, PropType U>
    using Template = And<T, U>;
    using TemplateArgs = std::tuple<P, Q>;

    // 以下略
};
```

テンプレートでないクラスについても、整合性をとるために引数が空のエイリアステンプレートと `std::tuple` を用意します。 `False` クラスを例にとると次のようになります。

```
class False final : PropBase {
public:
    template <class _ = void>
    using Template = False;
    using TemplateArgs = std::tuple<>;

    // 以下略
};
```

この前提の下で、 `ReplaceType<T, U, V>` は次のような関数として定義できます。

- `T` が `U` に一致しているならば `V` を返す。
- そうでなければ、 `T::TemplateArgs` が `std::tuple<Args...>` に一致するとして、
`T::Template<ReplaceType<Args[0], U, V>, ReplaceType<Args[1], U, V>, ...>` を返す。

実装は以下のようになります。

```
template <class T, class U, class V>
class ReplaceTypeImplementation;

template <template <class...> class Template, class TemplateArgs, class U, class V>
class ReplaceTypeHelper {};

template <template <class...> class Template, class U, class V, class... Args>
class ReplaceTypeHelper<Template, std::tuple<Args...>, U, V> {
public:
    using result = Template<typename ReplaceTypeImplementation<Args, U,
V::result...>;
};

template <class T, class U, class V>
class ReplaceTypeImplementation {
public:
    using result =
        std::conditional_t<
            std::same_as<T, U>,
            V,
```

```

        typename ReplaceTypeHelper<T::template Template, typename
T::TemplateArgs, U, V>::result
        >;
};

template <class T, class U, class V>
using ReplaceType = typename ReplaceTypeImplementation<T, U, V>::result;

```

ヘルパー関数 `ReplaceTypeHelper` では、テンプレートの特殊化を利用して `T::TemplateArgs` を `std::tuple<Args...>` にマッチさせ、`T::Template` の引数に `ReplaceType<Args, U, V>...` を代入しています。`ReplaceTypeImplementation` 本体では、`T` が `U` に一致するか確認した後、残りの処理を `ReplaceTypeHelper` に投げています。`ReplaceType<T, U, V>` は `ReplaceTypeImplementation<T, U, V>::type` のエイリアスです。

再帰の終了条件についてですが、`T` が `U` に一致しているとき `V` が返って終了するのはもちろん、`T` のテンプレート引数が空であるときも `T` 自身が返ることを容易に確認できます。

6.3. 基底クラス

命題クラスと同様に変数クラスにも基底クラスを用意しますが、命題クラスの場合と異なりオブジェクトに特別な意味がないので、以下のような簡単な実装になります。

```

class VarBase {};

template <class T>
concept VarType = std::is_base_of_v<VarBase, T>;

```

ついでに束縛変数の基底クラスと自由変数の基底クラスも作っておきます。

```

class BoundVarBase : VarBase {};

template <class T>
concept BoundVarType = std::is_base_of_v<BoundVarBase, T>;

class FreeVarBase : VarBase {};

template <class T>
concept FreeVarType = std::is_base_of_v<FreeVarBase, T>;

```

6.4. All

```

template <BoundVarType x, PropType P>
class All final : PropBase {
public:
    template <BoundVarType t, PropType T>
    using Template = All<t, T>;
    using TemplateArgs = std::tuple<x, P>;

```

```

consteval All(auto f) {
    auto p = f.template operator()<a>();
    static_assert(std::same_as<decltype(p), ReplaceType<P, x, a>>);
    initialized = true;
}

consteval All(const All& other) {
    initialized = other.initialized;
}

template <FreeVarType t>
consteval ReplaceType<P, x, t> elim() const {
    return PropBase::object<ReplaceType<P, x, t>>;
}

static void* operator new(size_t) = delete;
static void* operator new[](size_t) = delete;

private:
class a final : FreeVarBase {
public:
    template <class _ = void>
    using Template = a;
    using TemplateArgs = std::tuple<>;
    consteval a() {}
};

friend class PropBase;
consteval All() : initialized(true) {}

bool initialized;
};

```

All クラステンプレートは論理記号 $\forall$ に対応します。

コンストラクタ **All(auto f)** は $\forall$ 導入則を表しています。ラムダ式テンプレート **f** の引数に自由変数 **a** を代入して **ReplaceType<P, x, a>** のオブジェクトが返ってくるか検証します。

メンバ関数 **elim** は $\forall$ 除去則を表しています。任意の自由変数 **t** を受け取って **ReplaceType<P, x, t>** のオブジェクトが返します。

6.5. Exist

```

template <BoundVarType x, PropType P>
class Exist final : PropBase {
public:
    template <BoundVarType t, PropType T>
    using Template = Exist<t, T>;
    using TemplateArgs = std::tuple<x, P>;

    template <FreeVarType t>

```

```

consteval Exist(t, ReplaceType<P, x, t>) : initialized(true) {}

consteval Exist(const Exist& other) {
    initialized = other.initialized;
}

consteval auto elim(auto f) const {
    return f.template operator()<a>(PropBase::object<ReplaceType<P, x, a>>);
}

static void* operator new(size_t) = delete;
static void* operator new[](size_t) = delete;

private:
class a : FreeVarBase {
public:
    template <class _ = void>
    using Template = a;
    using TemplateArgs = std::tuple<>;
    consteval a() {}
};

friend class PropBase;
consteval Exist() : initialized(true) {}

bool initialized;
};

```

Exist クラステンプレートは論理記号 $\exists$ に対応します。

コンストラクタ `Exist(t, ReplaceType<P, x, t>)` は $\exists$ 導入則を表しています。適当な型 `t` のオブジェクトと `Replace<P, x, t>` のオブジェクトから `Exist<x, P>` のオブジェクトを生成できます。型 `t` をテンプレートとして渡していないのはC++の仕様上コンストラクタはテンプレート引数を明示的に指定できないからです。

メンバ関数 `elim` は $\exists$ 除去則を表しています。ラムダ式テンプレート `f` の引数に自由変数 `a` を代入したものに `ReplaceType<P, x, a>` のオブジェクトを渡して返ってきた値をそのまま返します。

6.6. Pred

```

template <size_t id, VarType... P>
class Pred final : PropBase {
public:
    template <VarType... Args>
    using Template = Pred<id, Args...>;
    using TemplateArgs = std::tuple<P...>;

    consteval Pred(const Pred& other) {
        initialized = other.initialized;
    }
}

```

```

static void* operator new(size_t) = delete;
static void* operator new[](size_t) = delete;

private:
    friend class PropBase;
    consteval Pred() : initialized(true) {}

    bool initialized;
};

template <size_t id>
using Prop = Pred<id>;

```

Pred クラステンプレートは述語 $P, Q, R, \dots$ に対応します。**id** に異なる非負整数を入れることで述語として区別します。(`sizeof...(P)` が異なるときも区別されます) **using** ディレクティブを用いて

```
template <class x, class y> using P = Prop<0, x, y>;
```

などのエイリアステンプレートを定義することを想定しています。

Prop は命題論理との互換性を保つために定義されたエイリアスです。

6.7. Bound

```

template <size_t id>
class Bound final : BoundVarBase {
public:
    template <class _ = void>
    using Template = Bound;
    using TemplateArgs = std::tuple<>;

    consteval Bound() {}
};

```

Bound クラステンプレートは束縛変数 $x, y, z, \dots$ に対応します。**id** に異なる非負整数を入れることで変数として区別します。(`sizeof...(P)` が異なるときも区別されます) **using** ディレクティブを用いて `using x = Bound<0>;` などのエイリアステンプレートを定義することを想定しています。

なお、自由変数についてはユーザー側が自由に定義できる必要性がない (**All** および **Exist** から渡されるもので事足りる) ので、**Free** クラステンプレートは作っていません。

6.8. 補足

6.8.1. 命題としての区別について

例えば $\forall x P(x)$ と $\forall y P(y)$ は数学では同じ命題を表しますが、上で述べた実装では **x** と **y** がクラスとして異なれば `All<x, P<x>>` と `Exist<y, P<y>>` は区別されるので注意してください。

6.8.2. 束縛変数に重複がある命題について

例えば `All<x, Exist<x, P<x>>>` のような束縛変数に重複がある命題は、コンストラクタを呼ぼうとするとちゃんとエラーになります。 `All` 内部で `ReplaceType<Exist<x, P<x>>, x, a>` が呼ばれますが、`Exist` の第一テンプレート引数は `x` に一致しているので自由変数 `a` を `Exist` の第一テンプレート引数に代入することになり、テンプレートパラメータ制約に違反するためです。

```
template <BoundVarType x, PropType P>
class Exist final : PropBase {
public:
    template <BoundVarType t, PropType T>
    using Template = Exist<t, T>; // ReplaceType内部でtを自由変数に置き換えようとして
    // エラーになる
    using TemplateArgs = std::tuple<x, P>;

    // 以下略
};
```

7. 述語論理の証明例

7.1. ド・モルガンの法則 (述語論理版)

$$\neg(\forall x P(x)) \leftrightarrow \exists x(\neg P(x))$$

```
template <class x, class P> using NotAll = Not<All<x, P>>;
template <class x, class P> using NotExist = Not<Exist<x, P>>;
template <class x> using P = Pred<0, x>;
template <class x> using NotP = Not<P<x>>;
using x = Bound<0>;

consteval Equiv<NotAll<x, P<x>>, Exist<x, NotP<x>>> solve() {
    return {
        [&](NotAll<x, P<x>> not_all_x_p_x) -> Exist<x, NotP<x>> {
            return Or<Exist<x, NotP<x>>, NotExist<x, NotP<x>>>().elim(
                [&](Exist<x, NotP<x>> exist_x_not_p_x) -> Exist<x, NotP<x>> {
                    return exist_x_not_p_x;
                },
                [&](NotExist<x, NotP<x>> not_exist_x_not_p_x) -> Exist<x, NotP<x>>
            );
        },
        [&](Exist<x, NotP<x>> exist_x_not_p_x) -> NotAll<x, P<x>> {
            return not_all_x_p_x(
                [&]<class a>() -> P<a> {
                    return Or<P<a>, NotP<a>>().elim(
                        [&](P<a> p_a) -> P<a> {
                            return p_a;
                        },
                        [&](NotP<a> not_p_a) -> NotP<a> {
                            return not_p_a;
                        }
                    );
                }
            );
        }
    };
}
```

```

    }
    );
}
).explode<Exist<x, NotP<x>>>();
}
);
},
[&](Exist<x, NotP<x>> exist_x_not_p_x) -> NotAll<x, P<x>> {
    return exist_x_not_p_x.elim(
        [&<class a>(NotP<a> not_p_a) -> NotAll<x, P<x>> {
            return [&](All<x, P<x>> all_x_p_x) -> False {
                return not_p_a(all_x_p_x.elim<a>());
            };
        }
    );
}
};
}

int main() {
    solve();
}

```

$$\neg(\exists x P(x)) \leftrightarrow \forall x(\neg P(x))$$

```

template <class x, class P> using NotAll = Not<All<x, P>>;
template <class x, class P> using NotExist = Not<Exist<x, P>>;
template <class x> using P = Pred<0, x>;
template <class x> using NotP = Not<P<x>>;
using x = Bound<0>;

consteval Equiv<NotExist<x, P<x>>, All<x, NotP<x>>> solve() {
    return {
        [&](NotExist<x, P<x>> not_exist_x_p_x) -> All<x, NotP<x>> {
            return [&<class a>() -> NotP<a> {
                return [&](P<a> p_a) -> False {
                    return not_exist_x_p_x({a(), p_a});
                };
            };
        },
        [&](All<x, NotP<x>> all_x_not_p_x) -> NotExist<x, P<x>> {
            return [&](Exist<x, P<x>> exist_x_p_x) -> False {
                return exist_x_p_x.elim(
                    [&<class a>(P<a> p_a) -> False {
                        return all_x_not_p_x.elim<a>()(p_a);
                    }
                );
            };
        }
    };
}

```

```
int main() {
    solve();
}
```

7.2. Exist と All の順序交換

$$\exists x \forall y P(x, y) \rightarrow \forall y \exists x P(x, y)$$

```
template <class x, class y> using P = Pred<0, x, y>;
using x = Bound<0>;
using y = Bound<1>;

consteval Impl<Exist<x, All<y, P<x, y>>>, All<y, Exist<x, P<x, y>>>> solve() {
    return [&](Exist<x, All<y, P<x, y>>> exist_x_all_y_p_x_y) -> All<y, Exist<x,
P<x, y>>> {
        return exist_x_all_y_p_x_y.elim(
            [&<class a>(All<y, P<a, y>> all_y_p_a_y) -> All<y, Exist<x, P<x, y>>>
{
                return [&<class b>() -> Exist<x, P<x, b>> {
                    return {a(), all_y_p_a_y.template elim<b>()};
                };
            }
        );
    };
}

int main() {
    solve();
}
```

7.3. Exist の And に対する分配

$$\exists x (P(x) \wedge Q(x)) \rightarrow \exists x P(x) \wedge \exists x Q(x)$$

```
template <class x> using P = Pred<0, x>;
template <class x> using Q = Pred<1, x>;
using x = Bound<0>;

Impl<Exist<x, And<P<x>, Q<x>>>, And<Exist<x, P<x>>, Exist<x, Q<x>>>> solve() {
    return [&](Exist<x, And<P<x>, Q<x>>> exist_x_and_p_x_q_x) -> And<Exist<x,
P<x>>, Exist<x, Q<x>>> {
        return exist_x_and_p_x_q_x.elim(
            [&<class a>(And<P<a>, Q<a>> and_p_a_q_a) -> And<Exist<x, P<x>>,
Exist<x, Q<x>>> {
                return {
                    {a(), and_p_a_q_a.left},
                    {a(), and_p_a_q_a.right}
                };
            }
        );
    };
}
```



```

    );
};

int main() {
    solve();
}

```

7.4. All の Or に対する分配

$$\forall x P(x) \vee \forall x Q(x) \rightarrow \forall x (P(x) \vee Q(x))$$

```

template <class x> using P = Pred<0, x>;
template <class x> using Q = Pred<1, x>;
using x = Bound<0>;

Impl<Or<All<x, P<x>>, All<x, Q<x>>>, All<x, Or<P<x>, Q<x>>>> solve() {
    return [&](Or<All<x, P<x>>, All<x, Q<x>>> or_all_x_p_x_all_x_q_x) -> All<x,
Or<P<x>, Q<x>>> {
        return [&]<class a>() -> Or<P<a>, Q<a>> {
            return or_all_x_p_x_all_x_q_x.elim(
                [&](All<x, P<x>> all_x_p_x) {
                    return Or<P<a>, Q<a>>(all_x_p_x.elim<a>());
                },
                [&](All<x, Q<x>> all_x_q_x) {
                    return Or<P<a>, Q<a>>(all_x_q_x.elim<a>());
                }
            );
        };
    };
}

int main() {
    solve();
}

```

8. 実装の全体像

完成した述語論理の形式的証明ツールを以下に示します。

※命題論理の実装と互換性があるので、4節で示した命題論理の証明例も以下と一緒にコンパイルできます。

```

#pragma once

#include <concepts>
#include <cstdint>
#include <type_traits>
#include <tuple>
#include <utility>

```

```

template <class T, class U, class V>
class ReplaceTypeImplementation;

template <template <class...> class Template, class TemplateArgs, class U, class
V>
class ReplaceTypeHelper {};

template <template <class...> class Template, class U, class V, class... Args>
class ReplaceTypeHelper<Template, std::tuple<Args...>, U, V> {
public:
    using result = Template<typename ReplaceTypeImplementation<Args, U,
V>::result...>;
};

template <class T, class U, class V>
class ReplaceTypeImplementation {
public:
    using result =
        std::conditional_t<
            std::same_as<T, U>,
            V,
            typename ReplaceTypeHelper<T::template Template, typename
T::TemplateArgs, U, V>::result
        >;
};

template <class T, class U, class V>
using ReplaceType = typename ReplaceTypeImplementation<T, U, V>::result;

class PropBase;

template <class P>
concept PropType = std::is_base_of_v<PropBase, P>;

class PropBase {
protected:
    constexpr PropBase() {}

    template <PropType P>
    static constexpr P object = P();
};

class VarBase {};

template <class T>
concept VarType = std::is_base_of_v<VarBase, T>;

class BoundVarBase : VarBase {};

template <class T>
concept BoundVarType = std::is_base_of_v<BoundVarBase, T>;

```

```

class FreeVarBase : VarBase {};

template <class T>
concept FreeVarType = std::is_base_of_v<FreeVarBase, T>;

class False final : PropBase {
public:
    template <class _ = void>
    using Template = False;
    using TemplateArgs = std::tuple<>;

    consteval False(const False& other) {
        initialized = other.initialized;
    }

    template <PropType P>
    consteval P explode() const {
        return PropBase::object<P>;
    }

    static void* operator new(size_t) = delete;
    static void* operator new[](size_t) = delete;

private:
    friend class PropBase;
    consteval False() : initialized(true) {}

    bool initialized;
};

template <PropType P>
class Not final : PropBase {
public:
    template <PropType T>
    using Template = Not<T>;
    using TemplateArgs = std::tuple<P>;

    consteval Not(auto f) {
        False q = f(PropBase::object<P>);
        initialized = true;
    }

    consteval Not(const Not& other) {
        initialized = other.initialized;
    }

    consteval False operator()(P) const {
        return PropBase::object<False>;
    }

    static void* operator new(size_t) = delete;
    static void* operator new[](size_t) = delete;

```

```

private:
    friend class PropBase;
    consteval Not() : initialized(true) {}

    bool initialized;
};

template <PropType P, PropType Q>
class And final : PropBase {
public:
    template <PropType T, PropType U>
    using Template = And<T, U>;
    using TemplateArgs = std::tuple<P, Q>;

    consteval And(P, Q) : initialized(true) {}
    consteval And(const And& other) {
        initialized = other.initialized;
    }

    const P left = PropBase::object<P>;
    const Q right = PropBase::object<Q>;

    static void* operator new(size_t) = delete;
    static void* operator new[](size_t) = delete;

private:
    friend class PropBase;
    consteval And() : initialized(true) {}

    bool initialized;
};

template <PropType P, PropType Q>
class Or final : PropBase {
public:
    template <PropType T, PropType U>
    using Template = Or<T, U>;
    using TemplateArgs = std::tuple<P, Q>;

    consteval Or() requires (std::same_as<Q, Not<P>> || std::same_as<P, Not<Q>>) :
initialized(true) {}
    consteval Or(P) : initialized(true) {}
    consteval Or(Q) requires (!std::same_as<P, Q>) : initialized(true) {}
    consteval Or(const Or& other) {
        initialized = other.initialized;
    }

    consteval auto elim(auto f, auto g) const {
        auto rf = f(PropBase::object<P>);
        auto rg = g(PropBase::object<Q>);
        static_assert(std::same_as<decltype(rf), decltype(rg)>);
    }
};

```

```

        return rf;
    }

    static void* operator new(size_t) = delete;
    static void* operator new[](size_t) = delete;

private:
    friend class PropBase;
    consteval Or() : initialized(true) {}

    bool initialized;
};

template <PropType P, PropType Q>
class Impl final : PropBase {
public:
    template <PropType T, PropType U>
    using Template = Impl<T, U>;
    using TemplateArgs = std::tuple<P, Q>;

    consteval Impl(auto f) {
        Q q = f(PropBase::object<P>);
        initialized = true;
    }
    consteval Impl(const Impl& other) {
        initialized = other.initialized;
    }

    consteval Q operator()(P) const {
        return PropBase::object<Q>;
    }

    static void* operator new(size_t) = delete;
    static void* operator new[](size_t) = delete;

private:
    friend class PropBase;
    consteval Impl() : initialized(true) {}

    bool initialized;
};

template <PropType P, PropType Q>
class Equiv final : PropBase {
public:
    template <PropType T, PropType U>
    using Template = Equiv<T, U>;
    using TemplateArgs = std::tuple<P, Q>;

    consteval Equiv(auto f, auto g) {
        Q q = f(PropBase::object<P>);
        P p = g(PropBase::object<Q>);
    }
};

```

```

        initialized = true;
    }
    consteval Equiv(const Equiv& other) {
        initialized = other.initialized;
    }

    consteval Q operator()(P) const {
        return PropBase::object<Q>;
    }

    consteval P operator()(Q) const requires (!std::same_as<P, Q>) {
        return PropBase::object<P>;
    }

    static void* operator new(size_t) = delete;
    static void* operator new[](size_t) = delete;

private:
    friend class PropBase;
    consteval Equiv() : initialized(true) {}

    bool initialized;
};

template <BoundVarType x, PropType P>
class All final : PropBase {
public:
    template <BoundVarType t, PropType T>
    using Template = All<t, T>;
    using TemplateArgs = std::tuple<x, P>;

    consteval All(auto f) {
        auto p = f.template operator()<a>();
        static_assert(std::same_as<decltype(p), ReplaceType<P, x, a>>);
        initialized = true;
    }
    consteval All(const All& other) {
        initialized = other.initialized;
    }

    template <FreeVarType t>
    consteval ReplaceType<P, x, t> elim() const {
        return PropBase::object<ReplaceType<P, x, t>>;
    }

    static void* operator new(size_t) = delete;
    static void* operator new[](size_t) = delete;

private:
    class a final : FreeVarBase {
    public:
        template <class _ = void>
        using Template = a;
    };

```

```

        using TemplateArgs = std::tuple<>;
        consteval a() {}
    };

    friend class PropBase;
    consteval All() : initialized(true) {}

    bool initialized;
};

template <BoundVarType x, PropType P>
class Exist final : PropBase {
public:
    template <BoundVarType t, PropType T>
    using Template = Exist<t, T>;
    using TemplateArgs = std::tuple<x, P>;

    template <FreeVarType t>
    consteval Exist(t, ReplaceType<P, x, t>) : initialized(true) {}

    consteval Exist(const Exist& other) {
        initialized = other.initialized;
    }

    consteval auto elim(auto f) const {
        return f.template operator()<a>(PropBase::object<ReplaceType<P, x, a>>);
    }

    static void* operator new(size_t) = delete;
    static void* operator new[](size_t) = delete;

private:
    class a : FreeVarBase {
    public:
        template <class _ = void>
        using Template = a;
        using TemplateArgs = std::tuple<>;
        consteval a() {}
    };

    friend class PropBase;
    consteval Exist() : initialized(true) {}

    bool initialized;
};

template <size_t id, VarType... P>
class Pred final : PropBase {
public:
    template <VarType... Args>
    using Template = Pred<id, Args...>;
    using TemplateArgs = std::tuple<P...>;

```

```
    consteval Pred(const Pred& other) {
        initialized = other.initialized;
    }

    static void* operator new(size_t) = delete;
    static void* operator new[](size_t) = delete;

private:
    friend class PropBase;
    consteval Pred() : initialized(true) {}

    bool initialized;
};

template <size_t id>
using Prop = Pred<id>;

template <size_t id>
class Bound final : BoundVarBase {
public:
    template <class _ = void>
    using Template = Bound;
    using TemplateArgs = std::tuple<>;

    consteval Bound() {}
};
```

9. アピールポイント

C++のテンプレート機能は非常に強力ですが、一方でラムダ式はHaskellなどの純粋関数型プログラミング言語に見られるものほど柔軟ではないため、形式的証明の構文に組み入れるのはかなり苦戦しました。しかし、C++のモダンな機能をフル活用し、なんとか述語論理の形式的証明までたどり着くことができました。本レポートを通して、C++のポテンシャルを再確認することができたと思います。